Repairing input range adaptors and `counted_iterator`

Document #:	P2259R0
Date:	2020-11-19
Project:	Programming Language C++
Audience:	LWG

Reply-to: Tim Song <t.canens.cpp@gmail.com>

Contents

§ 1 Abstract

This paper proposes fixes for several issues with `iterator_category` for range and iterator adaptors. This resolves [LWG3283], [LWG3289], and [LWG3408].

§ 2 The problem with `iterator_category`

This code does not compile:

```
std::vector<int> vec = {42};
auto r = vec | std::views::transform([](int c) { return std::views::single(c);})
             | std::views::join
             | std::views::filter([](int c) { return c > 0; });
r.begin();
```

Not because we are breaking any concept requirements:

— the `transform` produces a range of views of `int`;

— the `join` takes that and produce an input range of `int`;

— the `filter` should then produce another input range of `int`…in theory.

The problem is that `join_view`'s iterator for this case has a postfix `operator++` that returns `void`, making it not a valid C++17 iterator at all - even C++17 output iterators require `*i++` to be valid. In turn, that means that `iterator_traits<join_view::iterator>` is entirely empty, and `filter_view` cannot cope with that as currently specified, because it expects `iterator_category` to be always present (24.7.5.3 [range.filter.iterator]).

[LWG3283] and [LWG3289] were discussed at length during during the Belfast and Prague meetings. LWG was aware that as specified the range adaptors do not work with non-C++17 iterators due to these issues. However, I do not believe that it was clear to LWG that this impacts not just the one move-only iterator we have specified in the standard library, but also virtually *every* range adaptor with its own iterator type when used in conditions that produce an input range.

§ 2.1 The fix

We shouldn't (and can't) change postfix increment on the adaptor's iterators. There's nothing we can meaningfully return from `operator++` for arbitrary input iterators, especially if we are trying to adapt them. This was discussed in §3.1.2 of [P0541R1] and there is no need to rehash that discussion here.

These iterators are, then, not C++17 iterators *at all*, and specializations of `std::iterator_traits` for them have no members as currently specified. That seems to be an acceptable state of affairs.

It follows that the concern noted in [LWG3283]'s discussion is unlikely to be materialize in practice. While *some* C++20 input iterators might look like C++17 output iterators, I suspect that a large majority will not look like C++17 iterators at all. The fraction of input iterators that take advantage of the C++20 permission to not define `==` but not the permission to make postfix increment return `void` is likely small.

The fact that the `iterator_traits` specializations for these not-a-C++17-iterators don't have an `iterator_category` member (or any other) means that we don't really have a choice on the fix: we have to handle this case gracefully in the adaptors. The `output_iterator_tag` hack proposed in [LWG3289] is not a viable approach because these iterators aren't C++17 output iterators either, regardless of how hard we squint. Inventing a new `not_an_iterator_tag` defeats the purpose of C++20 `iterator_traits` as a C++17 compatibility layer: in C++17, if something is not an iterator, then `iterator_traits` is empty. We should keep this behavior, especially as the gain from making a new tag type is basically just somewhat simpler metaprogramming in the library.

As it turns out, the iterators of the range adaptors are only valid C++17 iterators (with a postfix increment that doesn't return `void`) when they are at least C++20 forward iterators or stronger. As all valid C++20 forward iterators meet the C++17 input iterator requirements, we can simply restrict the provision of `iterator_category` members to C++20 forward iterators. In those cases, the adapted iterators must also be C++20 forward iterators, so `iterator_category` should always be present in their `iterator_traits`, ignoring pathological program-defined specializations. That simplifies the wording considerably by removing the need to separately consider whether `iterator_category` is present.

§ 3 `counted_iterator` woes

As currently specified, the following test case fails (this is [LWG3408]):

```
auto v = views::iota(0);
auto i = counted_iterator{v.begin(), 5};
static_assert(random_access_iterator<decltype(i)>);
```

Additionally, we can't even ask the question whether `counted_iterator<int*>` is a `contiguous_iterator`:

```
static_assert(contiguous_iterator<counted_iterator<int*>> || true);  // hard error
```

The problem here is that `counted_iterator` tries to emulate the behavior of the iterator it wraps (with the notable exception of `->`) by defining an `iterator_traits` specialization:

```
template<input_iterator I>
struct iterator_traits<counted_iterator<I>> : iterator_traits<I> {
  using pointer = void;
};
```

But `iterator_traits` plays two very different roles in the C++20 iterator design:

1. When generated from the primary template, it serves as a C++17 compatibility layer. Notably, we never define an `iterator_concept` member in this case, only `iterator_category`, and the latter is derived from the type's C++17-iterator-ness.
2. When explicitly or partially specialized, it serves as the non-intrusive customization point for iterators that takes precedence over member types. In particular, if `iterator_concept` is not defined, then we use `iterator_category` to cap the type's C++20-iterator-ness.

The problem with the `iterator_traits` partial specialization above is that it takes an `iterator_traits` being used for (1) and uses its contents for case (2). In the [LWG3408] example, `iota_view<...>::iterator` properly reported that it is a *C++17* input iterator, but the `counted_iterator` specialization means that we took that as saying that `counted_iterator<iota_view<...>::iterator>` is only a *C++20* input iterator.

Moreover, `counted_iterator` fails to handle wrapping `contiguous_iterators` correctly: by inheriting from an `iterator_traits` specialization, it can inherit the `contiguous_iterator_tag` opt-in (e.g., for pointers), but it neither defines `operator->` nor `pointer_traits::to_address`, so `std::to_address(ci)` is ill-formed, and that function template uses a deduced return type, so the error happens outside the immediate context. As a result, even asking the question is ill-formed.

§ 3.1 Fixing `counted_iterator`

This paper proposes fixing `counted_iterator` as follows:

— Constrain the `iterator_traits` specialization so that it is only used when `iterator_traits<I>` is not generated from the primary template. In other words, only provide the specialization for case (2) when we are already there.

— Provide `value_type` and `difference_type` member typedefs, as this is the usual way to providing these types in C++20 when there is no `iterator_traits` specialization.

— Remove the `incrementable_traits` specialization, since that doesn't add anything when we are defining a `difference_type` member.

— Provide member `iterator_concept` and `iterator_category` when the wrapped iterator type provides them, to honor its opt-in/opt-outs.

— Provide member `operator->` if the wrapped iterator is contiguous, so that `std::to_address` works and allows `counted_iterator` to be contiguous itself if the wrapped iterator is also contiguous. The definition of `pointer` in the `iterator_traits` specialization needs to be adjusted accordingly.

§ 4 Implementation experience

The wording in this paper has been implemented atop current libstdc++ master and passes all existing tests, and has been confirmed to fix the examples given above.

§ 5 Wording

This wording is relative to [N4868].

§ 5.1 `iterator_category`

1. Add an example to 23.3.2.3 [iterator.traits] p4 as indicated:

> 4 Explicit or partial specializations of `iterator_traits` may have a member type `iterator_concept` that is used to indicate conformance to the iterator concepts (23.3.4 [iterator.concepts]). [*Example ?:* To indicate conformance to the `input_iterator` concept but a lack of conformance to the *Cpp17InputIterator* requirements (23.3.5.3 [input.iterators]), an `iterator_traits` specialization might have `iterator_concept` denote `input_iterator_tag` but not define `iterator_category`. — *end example*]

2. Edit 23.5.3.2 [move.iterator] as indicated:

```
namespace std {
  template<class Iterator>
  class move_iterator {
  public:
    using iterator_type     = Iterator;
    using iterator_concept  = input_iterator_tag;
    using iterator_category = see below;    // not always present
    using value_type        = iter_value_t<Iterator>;
    using difference_type   = iter_difference_t<Iterator>;
    using pointer           = Iterator;
    using reference         = iter_rvalue_reference_t<Iterator>;

    // [...]
  };
}
```

1 The member *typedef-name* `iterator_category` is defined if and only if the *qualified-id* `iterator_traits<Iterator>::iterator_category` is valid and denotes a type. In that case, `iterator_category` denotes

(1.1) — `random_access_iterator_tag` if the type `iterator_traits<Iterator>::iterator_category` models `derived_from<random_access_iterator_tag>`, and

(1.2) — `iterator_traits<Iterator>::iterator_category` otherwise.

3. Edit 23.5.4.2 [common.iter.types]:

[Drafting note: `common_iterator` is a C++17 compatibility layer, and that's reflected in its synthesis of an `operator==` even when the underlying type does not support it. The proposed wording here follows that design by similarly providing a C++17-conforming postfix `operator++` even if the underlying type doesn't.]

1 The nested *typedef-name*s of the specialization of `iterator_traits` for `common_iterator<I, S>` are defined as follows.

(1.1) — `iterator_concept` denotes `forward_iterator_tag` if `I` models `forward_iterator`; otherwise it denotes `input_iterator_tag`.

(1.2) — `iterator_category` denotes `forward_iterator_tag` if the *qualified-id* `iterator_traits<I>::iterator_category` is valid and denotes a type that models `derived_from<forward_iterator_tag>`; otherwise it denotes `input_iterator_tag`.

(1.3) — If the expression `a.operator->()` is well-formed, where `a` is an lvalue of type `const common_iterator<I, S>`, then `pointer` denotes the type of that expression. Otherwise, `pointer` denotes `void`.

4. Edit 23.5.4.5 [common.iter.nav] p5 as indicated:

```
decltype(auto) operator++(int);
```

4 *Preconditions:* `holds_alternative<I>(v_)`.

5 *Effects:* If `I` models `forward_iterator`, equivalent to:

```
common_iterator tmp = *this;
++*this;
return tmp;
```

Otherwise, if `requires (I& i) { { *i++ } -> can-reference; }` is `true` or `constructible_from<iter_value_t<I>, iter_reference_t<I>>` is `false`, equivalent to: `return get<I>(v_)++;`

Otherwise, equivalent to:

```
postfix-proxy p(**this);
++*this;
return p;
```

where *postfix-proxy* is the exposition-only class:

```
class postfix-proxy  {
  iter_value_t<I> keep_;
  postfix-proxy(iter_reference_t<I>&& x)
    : keep_(std::move(x)) {}
public:
  const iter_value_t<I>& operator*() const {
    return keep_;
  }
};
```

5. Edit 24.6.4.3 [range.iota.iterator] as indicated:

[Drafting note: While this member typedef is arguably harmless, we should avoid providing misleading tags for something that is, in fact, not a C++17 iterator.]

```
namespace std::ranges {
  template<weakly_incrementable W, semiregular Bound>
    requires weakly-equality-comparable-with<W, Bound>
  struct iota_view<W, Bound>::iterator {
  private:
    W value_ = W();             // exposition only
  public:
    using iterator_concept = see below;
    using iterator_category = input_iterator_tag; // present only if W
        models incrementable
    using value_type = W;
    using difference_type = IOTA-DIFF-T(W);

    // [...]
  };
}
```

6. Edit 24.7.5.3 [range.filter.iterator] as indicated:

[Drafting note: All C++20 forward iterators should qualify as C++17 input iterators, and so the wording expects `iterator_category` to be present. It seems unnecessary to accommodate hypothetical types that artificially define away C++17 iterator-ness.]

```
namespace std::ranges {
  template<input_range V, indirect_unary_predicate<iterator_t<V>> Pred>
    requires view<V> && is_object_v<Pred>
  class filter_view<V, Pred>::iterator {
  private:
    iterator_t<V> current_ = iterator_t<V>();   // exposition only
    filter_view* parent_ = nullptr;             // exposition only
  public:
    using iterator_concept  = see below;
    using iterator_category = see below;        // not always present
    using value_type        = range_value_t<V>;
    using difference_type   = range_difference_t<V>;

    // [...]
```

```
  };
}
```

[…]

3 The member *typedef-name* `iterator_category` is defined if and only if `V` models `forward_range`. In that case, *`iterator`*`::iterator_category` is defined as follows:

(3.1) — Let `C` denote the type `iterator_traits<iterator_t<V>>::iterator_category`.

(3.2) — If `C` models `derived_from<bidirectional_iterator_tag>`, then `iterator_category` denotes `bidirectional_iterator_tag`.

(3.3) — Otherwise, if `C` models `derived_from<forward_iterator_tag>`, then `iterator_category` denotes `forward_iterator_tag`.

(3.4) — Otherwise, `iterator_category` denotes `C`.

7. Edit 24.7.6.3 [range.transform.iterator] as indicated:

```
namespace std::ranges {
  template<input_range V, copy_constructible F>
    requires view<V> && is_object_v<F> &&
             regular_invocable<F&, range_reference_t<V>> &&
             can-reference<invoke_result_t<F&, range_reference_t<V>>>
  template<bool Const>
  class transform_view<V, F>::iterator {
  private:
    using Parent =                                  // exposition only
      conditional_t<Const, const transform_view, transform_view>;
    using Base   =                                  // exposition only
      conditional_t<Const, const V, V>;
    iterator_t<Base> current_ =                     // exposition only
      iterator_t<Base>();
    Parent* parent_ = nullptr;                      // exposition only
  public:
    using iterator_concept  = see below;
    using iterator_category = see below;        // not always present
    using value_type        =
      remove_cvref_t<invoke_result_t<F&, range_reference_t<Base>>>;
    using difference_type   = range_difference_t<Base>;

    // [...]
  };
}
```

[…]

2 The member *typedef-name* `iterator_category` is defined if and only if *`Base`* models `forward_range`. In that case, `iterator::iterator_category` is defined as follows: Let `C` denote the type `iterator_traits<iterator_t<`*`Base`*`>>::iterator_category`.

(2.1) — If `is_lvalue_reference_v<invoke_result_t<F&, range_reference_t<Base>>>` is `true`, then

(2.1.1) — if `C` models `derived_from<contiguous_iterator_tag>`, iterator_category denotes `random_access_iterator_tag`;

(2.1.2) — otherwise, `iterator_category` denotes `C`.

(2.2) — Otherwise, `iterator_category` denotes `input_iterator_tag`.

8. Edit 24.7.11.3 [range.join.iterator] as indicated:

```
namespace std::ranges {
  template<input_range V>
    requires view<V> && input_range<range_reference_t<V>> &&
             (is_reference_v<range_reference_t<V>> ||
              view<range_value_t<V>>)
  template<bool Const>
  struct join_view<V>::iterator {
  private:
    using Parent =                                              //
        exposition only
      conditional_t<Const, const join_view, join_view>;
    using Base   = conditional_t<Const, const V, V>;            //
        exposition only

    static constexpr bool ref-is-glvalue =                      //
        exposition only
      is_reference_v<range_reference_t<Base>>;

    // [...]

  public:
    using iterator_concept  = see below;
    using iterator_category = see below;        // not always present
    using value_type        = range_value_t<range_reference_t<Base>>;
    using difference_type   = see below;

    // [...]
  };
}
```

[...]

2 The member *typedef-name* `iterator_category` is defined if and only if *`ref-is-glvalue`* is `true`, *`Base`* models `forward_range`, and `range_reference_t<Base>` models `forward_range`. In that case, `iterator::iterator_category` is defined as follows:

(2.1) — Let *OUTERC* denote `iterator_traits<iterator_t<Base>>::iterator_category`, and let *INNERC* denote `iterator_traits<iterator_t<range_reference_t<Base>>>::iterator_category`.

(2.2) — If *`ref-is-glvalue`* is `true` and *OUTERC* and *INNERC* each model `derived_from<bidirectional_iterator_tag>`, `iterator_category` denotes `bidirectional_iterator_tag`.

(2.3) — Otherwise, if *ref-is-glvalue* is `true` and *OUTERC* and *INNERC* each model `derived_from<forward_iterator_tag>`, `iterator_category` denotes `forward_iterator_tag`.

(2.4) — Otherwise, if *OUTERC* and *INNERC* each model `derived_from<input_iterator_tag>`, `iterator_category` denotes `input_iterator_tag`.

(2.5) — Otherwise, `iterator_category` denotes `output_iterator_tag`.

9. Edit 24.7.12.3 [range.split.outer] as indicated:

```
namespace std::ranges {
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
         ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  template<bool Const>
  struct split_view<V, Pattern>::outer-iterator {
  private:
    using Parent =                          // exposition only
      conditional_t<Const, const split_view, split_view>;
    using Base   =                          // exposition only
      conditional_t<Const, const V, V>;

      // [...]
  public:
    using iterator_concept  =
      conditional_t<forward_range<Base>, forward_iterator_tag,
        input_iterator_tag>;
    using iterator_category = input_iterator_tag; // present only if Base
        models forward_range

    // [...]
  };
}
```

10. Edit 24.7.12.5 [range.split.inner] as indicated:

```
namespace std::ranges {
  template<input_range V, forward_range Pattern>
    requires view<V> && view<Pattern> &&
             indirectly_comparable<iterator_t<V>, iterator_t<Pattern>,
         ranges::equal_to> &&
             (forward_range<V> || tiny-range<Pattern>)
  template<bool Const>
  struct split_view<V, Pattern>::inner-iterator {
  private:
    using Base = conditional_t<Const, const V, V>;      // exposition only

    // [...]

  public:
    using iterator_concept  = typename outer-
        iterator<Const>::iterator_concept;
```

```
    using iterator_category = see below;                // present only if
        Base models forward_range

    // [...]
  };
}
```

1 The *typedef-name* `iterator_category` denotes:

(1.1) — `forward_iterator_tag` if `iterator_traits<iterator_t<Base>>::iterator_category` models `derived_from<forward_iterator_tag>`;

(1.2) — otherwise, `iterator_traits<iterator_t<Base>>::iterator_category`.

11. Edit 24.7.16.3 [range.elements.iterator] as indicated:

```
namespace std::ranges {
  template<input_range V, size_t N>
    requires view<V> && has-tuple-element<range_value_t<V>, N> &&
             has-tuple-element<remove_reference_t<range_reference_t<V>>, N>
  template<bool Const>
  class elements_view<V, N>::iterator {                 // exposition only
    using Base = conditional_t<Const, const V, V>;      // exposition only

    iterator_t<Base> current_ = iterator_t<Base>();
  public:
    using iterator_category = typename
        iterator_traits<iterator_t<Base>>::iterator_categorysee below; //
        not always present
    using value_type = remove_cvref_t<tuple_element_t<N,
        range_value_t<Base>>>;
    using difference_type = range_difference_t<Base>;

    // [...]
  };
}
```

? The member *typedef-name* `iterator_category` is defined if and only if *`Base`* models `forward_range`. In that case, `iterator_category` denotes:

(?.1) — `input_iterator_tag` if `get<N>(*current_)` is an rvalue;

(?.2) — otherwise, `iterator_traits<iterator_t<Base>>::iterator_category`.

§ 5.2 `counted_iterator`

1. Edit 23.2 [iterator.synopsis], Header `<iterator>` synopsis, as indicated:

```
#include <compare>              // see [compare.syn]
#include <concepts>             // see [concepts.syn]

namespace std {

  // [...]
```

```
  template<class I>
    struct incrementable_traits<counted_iterator<I>>;

  template<input_iterator I>
    requires see below
  struct iterator_traits<counted_iterator<I>>;

  // [...]
}
```

2. Edit the definition of `iterator_traits<counted_iterator<I>>` in 23.5.6.1 [counted.iterator] as indicated:

```
    template<input_iterator I>
+     requires same_as<ITER_TRAITS(I), iterator_traits<I>>   // see
        23.3.4.1 [iterator.concepts.general]
    struct iterator_traits<counted_iterator<I>> : iterator_traits<I> {
-     using pointer = void;
+     using pointer = conditional_t<contiguous_iterator<I>,
        add_pointer_t<iter_reference_t<I>>, void>;
  };
```

3. Edit the definition of `counted_iterator` in 23.5.6.1 [counted.iterator] as indicated:

```
namespace std {
  template<input_or_output_iterator I>
  class counted_iterator {
  public:
    using iterator_type = I;
+   using value_type = iter_value_t<I>;                          // present
      only if I models indirectly_readable
+   using difference_type = iter_difference_t<I>;
+   using iterator_concept = typename I::iterator_concept;    // present
      only if the qualified-id
+                                                               //
      I::iterator_concept is valid and denotes a type
+   using iterator_category = typename I::iterator_category;  // present
      only if the qualified-id
+                                                               //
      I::iterator_category is valid and denotes a type

    // [...]

    constexpr I base() const & requires copy_constructible<I>;
    constexpr I base() &&;
    constexpr iter_difference_t<I> count() const noexcept;
    constexpr decltype(auto) operator*();
    constexpr decltype(auto) operator*() const
      requires dereferenceable<const I>;

+   constexpr auto operator->() const noexcept
+     requires contiguous_iterator<I>;

    // [...]
  };
}
```

4. Strike the `incrementable_traits<counted_iterator<I>>` specialization in 23.5.6.1 [counted.iterator] as unnecessary:

```
~~template<class I>~~
~~struct incrementable_traits<counted_iterator<I>> {~~
~~  using difference_type = iter_difference_t<I>;~~
~~};~~
```

5. Add the following to 23.5.6.4 [counted.iter.elem]:

```
constexpr auto operator->() const noexcept
  requires contiguous_iterator<I>;

  ?  Effects: Equivalent to: return to_address(current);
```

§ 6 References

[LWG3283] Eric Niebler. Types satisfying input_iterator but not equality_comparable look like C++17 output iterators.
https://wg21.link/lwg3283

[LWG3289] Eric Niebler. Cannot opt out of C++17 iterator-ness without also opting out of C++20 iterator-ness.
https://wg21.link/lwg3289

[LWG3408] Patrick Palka. LWG 3291 reveals deficiencies in counted_iterator.
https://wg21.link/lwg3408

[N4868] Richard Smith. 2020. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4868

[P0541R1] Eric Niebler. 2017. Ranges TS: Post-Increment on Input and Output Iterators.
https://wg21.link/p0541r1